

Hasan Zaidi
ITAI 3377
Prof. Ayyagari
February 20, 2026

Reflective Journal: IIoT Protocols Project

1. Introduction

For this project, I designed and simulated an Industrial Internet of Things (IIoT) sensor network to see how different communication protocols handle real-time data. My main goal was to move beyond just reading about these technologies and actually see them in action by generating live temperature and humidity readings. I expected to run into some setup issues, but I was looking forward to see how a live dashboard could visualize data coming from different sources simultaneously.

2. Personal Contributions

I was responsible for setting up the development environment, which included creating a Python virtual environment and managing dependencies like paho-mqtt and aiocoap. I wrote and configured the simulation scripts for MQTT, CoAP, and OPC UA, ensuring each one correctly generated random sensor values every second. Additionally, I built the data visualization script using Matplotlib and organized the final project repository on GitHub, making sure all my findings were documented in the comparison report.

3. Learning Outcomes

This hands-on experience taught me the practical differences between the three protocols. I learned that MQTT is extremely efficient for dashboards because of its broker-based publish/subscribe model. CoAP stood out as a lightweight option for small devices because it uses UDP and confirms delivery with "ACK" messages. Finally, I saw that OPC UA is the most complex and secure "heavyweight" standard, which I noticed through the encryption and security requirements during the server setup. My biggest takeaway is that there isn't a best protocol; you have to choose based on whether you need speed, low power, or high security.

4. Challenges and Solutions

I hit a major wall when trying to get the real-time graph to work on my Mac; the script kept crashing because I was trying to update the GUI from the wrong thread. To fix this, I had to restructure the code to handle data processing in the background while keeping the plotting on the main thread. I also struggled with the CoAP simulation because it was refusing connections. I realized that I had to run a separate CoAP server script to actually receive the data before the

client could send anything. These bugs taught me a lot about how network timing and local hosting actually work.

5. Future Applications

I can definitely see myself using these skills in future automation or smart-home projects where I need to connect different devices reliably. Understanding the "under the hood" mechanics of protocols like OPC UA will be a huge advantage if I ever work in industrial manufacturing or systems integration. If I were to take this project further, I'd try to get all three protocols feeding into a single unified dashboard or try running the sensors on actual hardware like a Raspberry Pi to see how they perform under real-world network stress.